Chapter - 1

PRINCIPLES OF OBJECT ORIENTED PROGRAMMING

Software Crisis

- Developments in software technology continue to be dynamic.
- New tools and techniques are announced in quick succession.
- The following issues need to be addressed to face this crisis:
  - How to represent real-life entities of problems in system design?
  - How to design systems with open interfaces?
  - How to ensure reusability and extensibility of modules?
  - How to develop modules that are tolerant to any changes in future?
  - How to improve software productivity and decrease software cost?
  - How to improve the quality of software?
  - How to manage time schedules?
  - How to industrialize the software development process?

Software Crisis

- These studies and other reports on software implementation suggest that software products should be evaluated carefully for their quality before they are delivered and implemented.
- Some of the quality issues that must be considered for critical evaluation are:
  1. Correctness
  2. Maintainability
  3. Reusability
  4. Openness and interoperability
  5. Portability
  6. Security
  7. Integrity
  8. User friendliness

Software Crisis

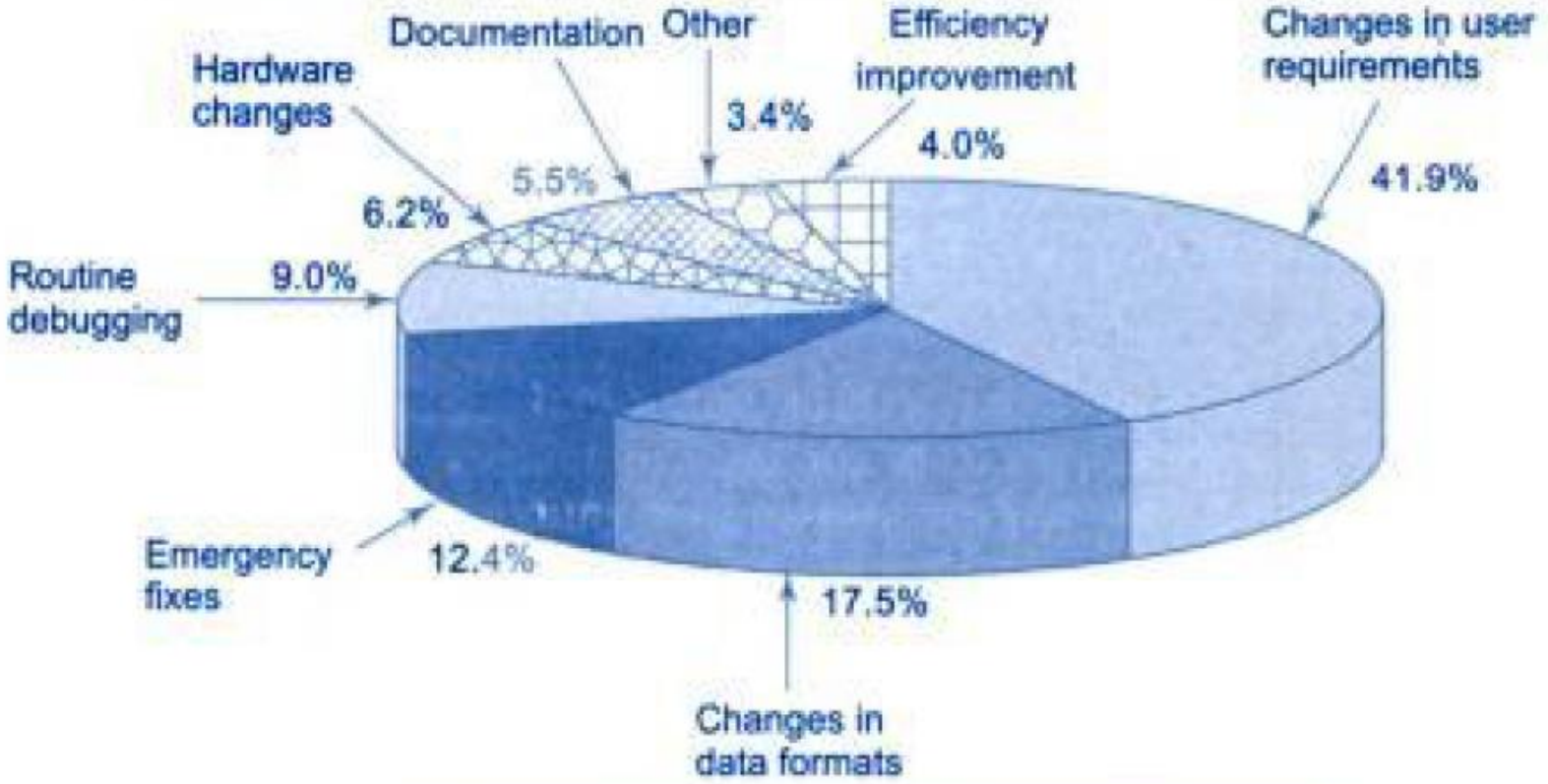


Fig. 1.2 ⇔ *Breakdown of maintenance costs*

Software Evolution

- Ernest Tello, a well-known writer in the field of artificial intelligence, compared the evolution of software technology to the growth of a tree. Like a tree, the software evolution has had distinct phases or "layers" of growth. These layers were built up one by one over the last five decades as shown in Fig. (next slide), with each layer representing an improvement over the previous one. However, the analogy fails if we consider the life of these layers. In software systems, each of the layers continues to be functional, whereas in the case of trees, only the uppermost layer is functional.
- Since the invention of the computer, many programming approaches have been tried.
- These include techniques such as modular programming, top-down programming, bottom- up programming and structured programming.

Software Evolution

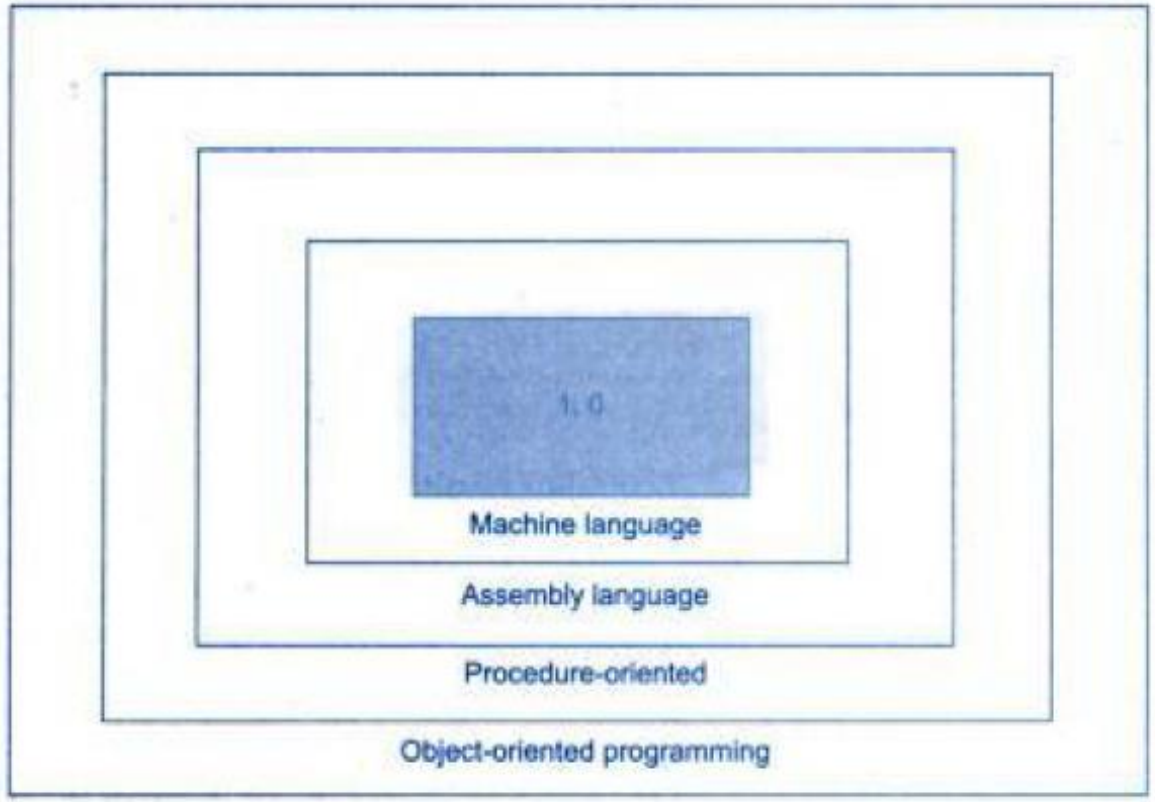


Fig. 1.3 ⇔ *Layers of computer software*

Software Evolution

- With the advent of languages such as C, structured programming became very popular and was the main technique of the 1980s. Structured programming was a powerful tool that enabled programmers to write moderately complex programs fairly easily. However, as the programs grew larger, even the structured approach failed to show the desired results in terms of bug-free, easy-to-maintain, and reusable programs.
- Object-Oriented Programming (OOP) is an approach to program organization and development that attempts to eliminate some of the pitfalls of conventional programming methods by incorporating the best of structured programming features with several powerful new concepts.

Procedure-Oriented Programming

- Procedure Oriented Programming (POP) contains step by step procedure to execute. Here, the problems get decomposed into small parts and then to solve each part one or more functions are used.
- Commonly known POP languages: COBOL, FORTRAN, C
- Thus in POP approach, the problem is viewed as a sequence of things to be done, such as, input taking, calculating and displaying.
- The primary focus stays on functions which will be used to accomplish each task.
- In POP approach, we write lines of instruction to be followed by computer to accomplish a task and by organizing these instructions into groups we create functions.

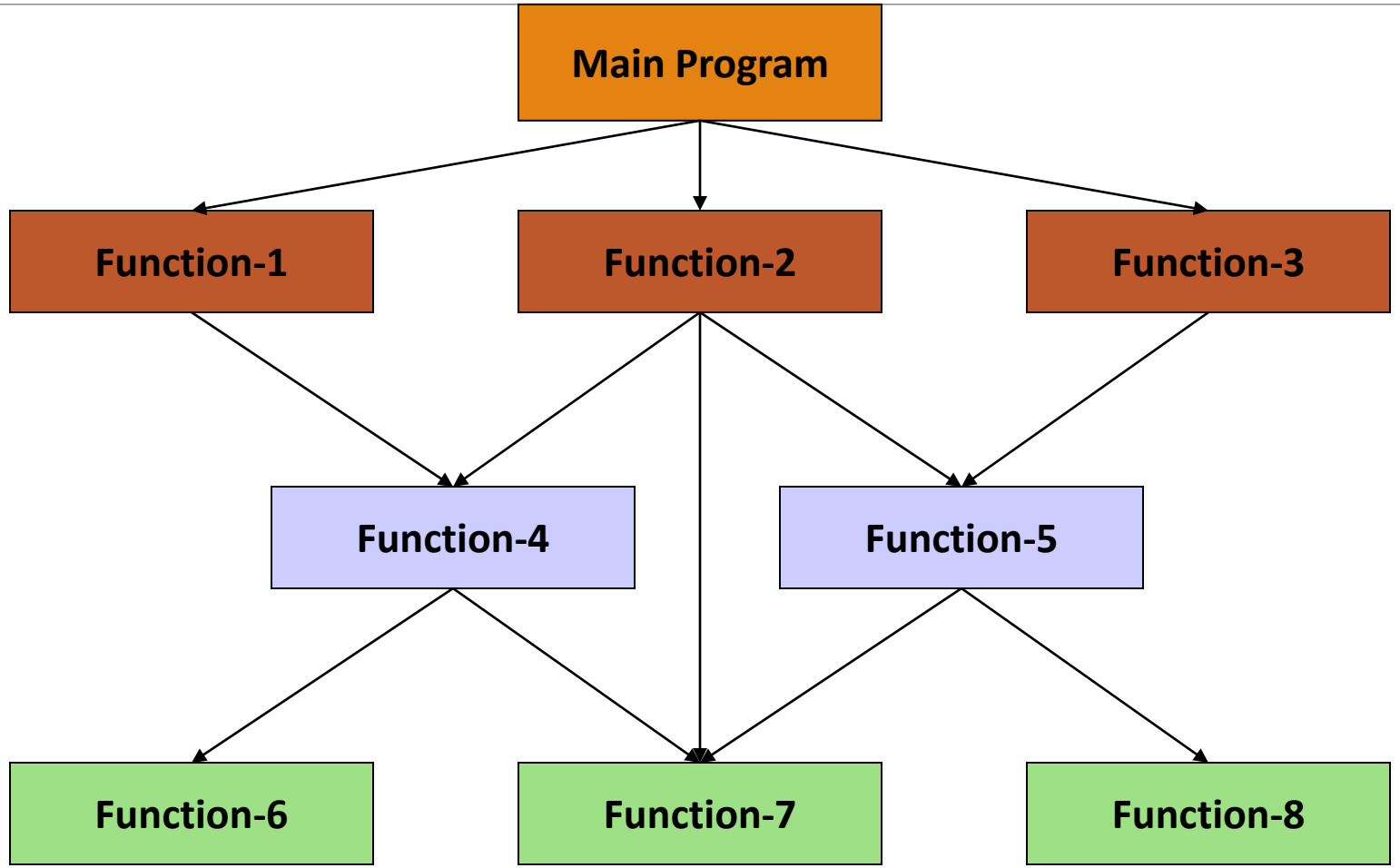
Procedure-Oriented Programming

continue …

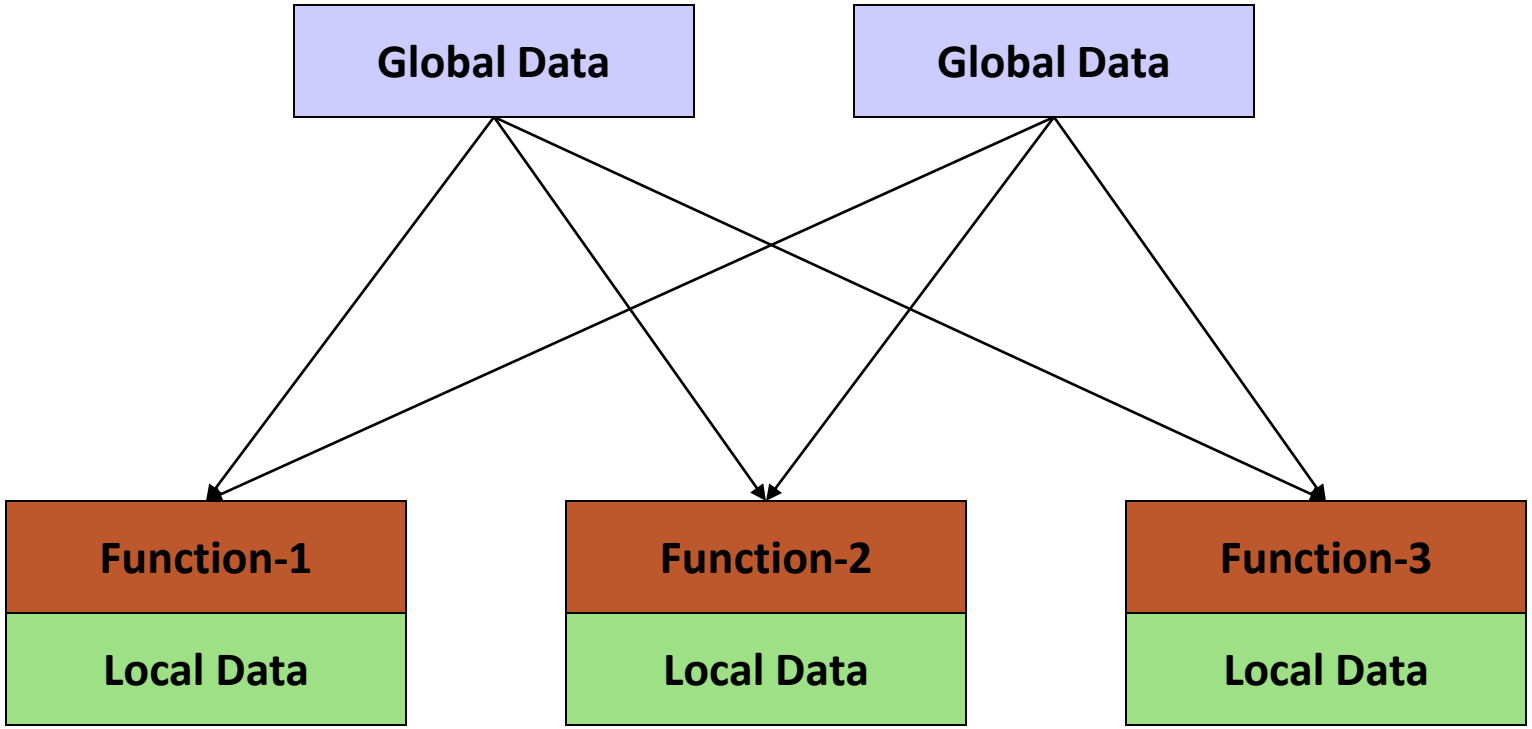
For example, considering that we have to take two values from user and then to display the summation of the inputted values. In POP approach this problem may be decomposed as following,

- First take an input and put it under one variable, here taking as a.
- Then take another input and put it under another variable, here taking as b.
- Now define a variable, such as c, as c = a + b.
- Now display c.

Typical structure of procedure-oriented program



Relationship of data and functions in procedural programming



Characteristics of Procedure-Oriented Programming

- Emphasis is on doing things (algorithms).
- Large programs are divided into smaller programs known as functions.
- Most of the functions share global data.
- Data move openly around the system from function to function.
- Functions transform data from one form to another.
- Employs top-down approach in program design.

Problems/Drawbacks of POP

It is true that the programs created using *POP* can be extremely efficient and high-performance, but the main problem is the maintenance. Some other problems includes the followings,

- Insecure data, if we declare a variable before main function then it can be accessed freely from any function present in the program.
- No better memory management.
- Difficult to implement today's client requirements.
- No structure or code reusability. Hence time of development, testing and length of program increases.
- As length of application increases it causes slow performance.
- No proper way method for Exception handling.

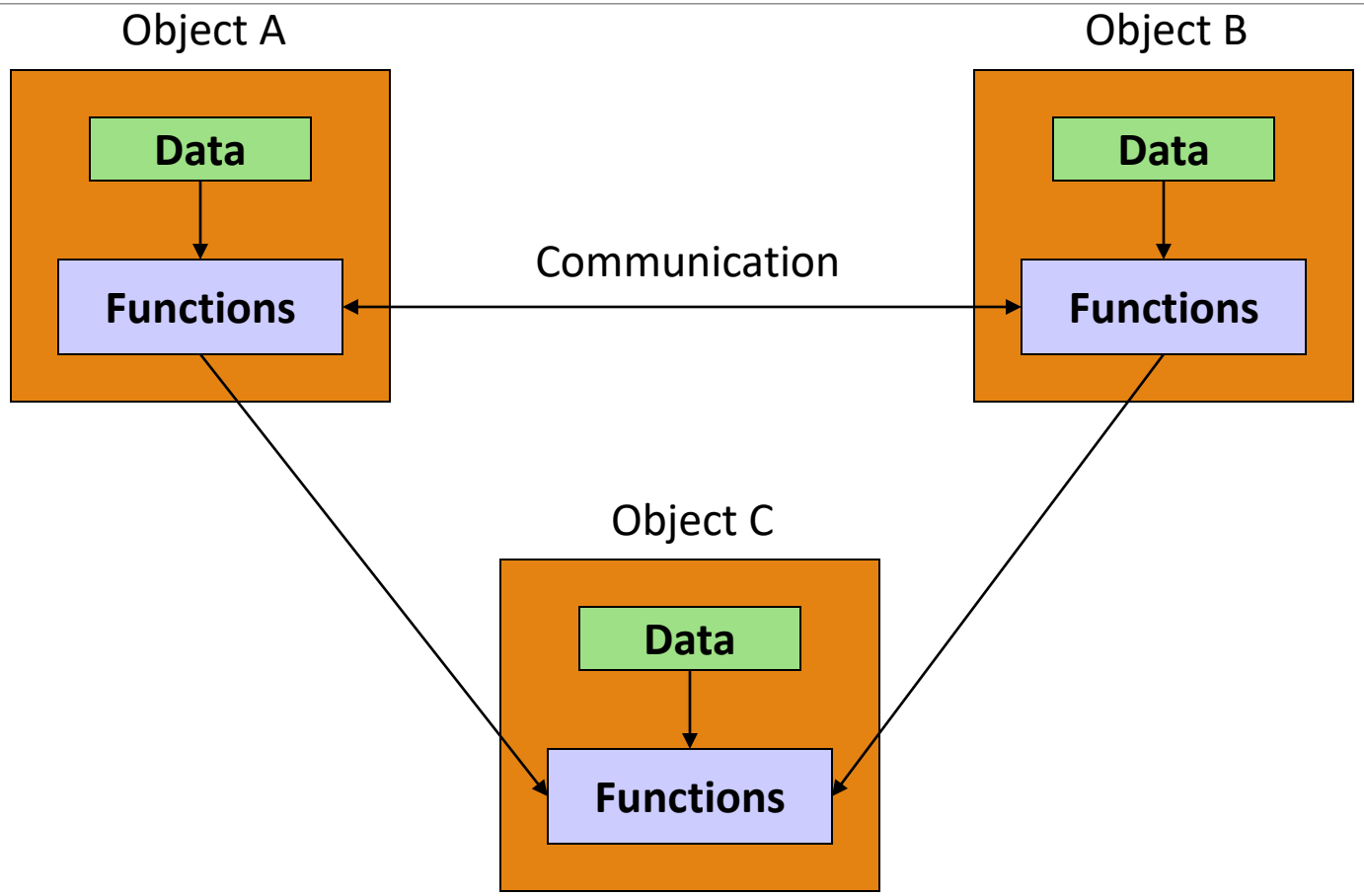
Solution to Problems of POP Is in OOP

Object-Oriented Programming

Object-oriented programming (OOP) is a programming language model organized around objects rather than "actions" and data rather than logic.

The **OOP** is designed on a certain concept to reach its goal of overcoming the drawbacks of conventional programming approach.

Continued …


Object A
Data
Functions
Communication
Object B
Data
Functions
Object C
Data
Functions

Object-Oriented Programming

Some of the striking features of object-oriented programming are:

- Emphasis is on data rather than procedure.
- Programs are divided into what are known as objects.
- Data structures are designed such that they characterise the objects.
- Functions that operate on the data of an object are tied together in the data structure.
- Data is hidden and cannot be accessed by external functions.
- Objects may communicate with each other through functions. New data and functions can be easily added whenever necessary.
- Follows bottom-up approach in program design.

Basic Concepts of Object-Oriented Programming

- Objects
- Classes
- Data Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing

Basic Concepts of OOP

Object

Objects are the basic identifiable runtime entities in object-oriented programming.

An object has its name and own properties. (i.e. Object contains data or information.)

For example- apple is an object and its properties are name, color, size and taste etc, like:

- Name - apple
- Color - red
- Size - medium in size
- Taste - sweet

The above mentioned example demonstrates that **each and every thing in real world exists in the form of object. It is also clarify that each and every object belongs to a class**.

For example: **an apple is an object of class fruit**.

Continued …

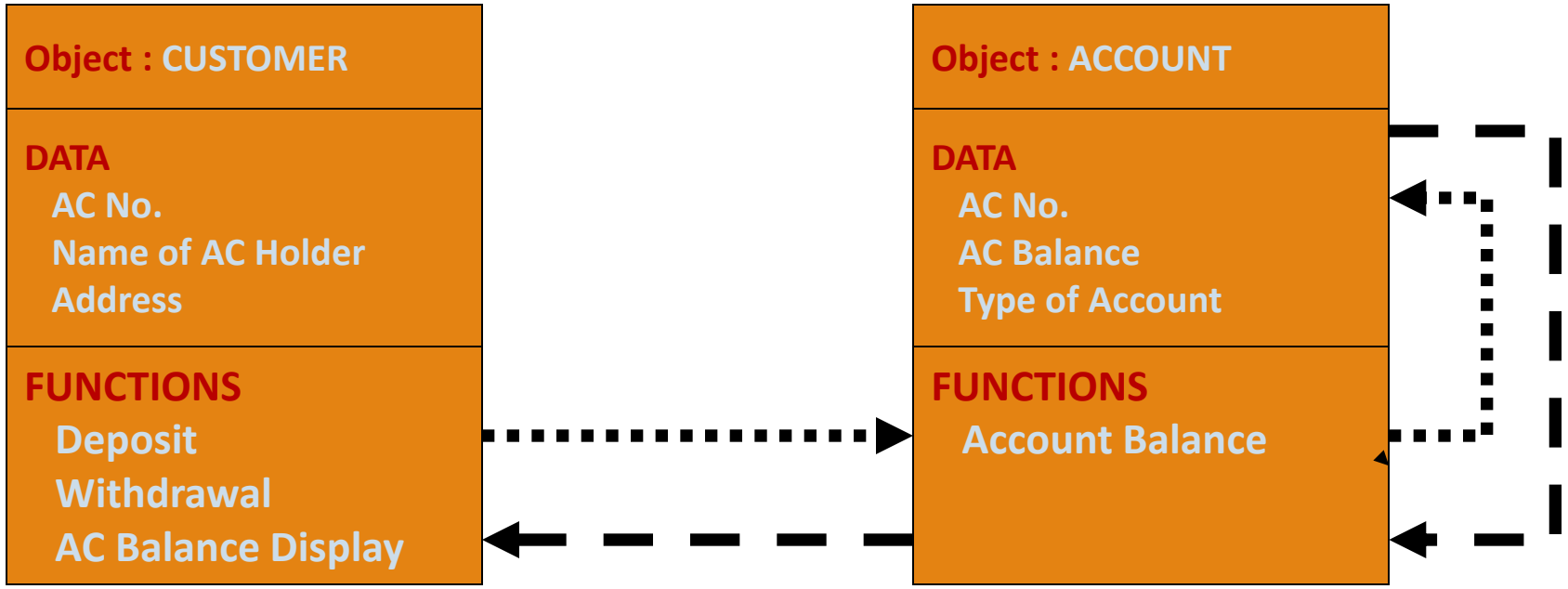
An Object contains both the data and the function, which operates on the data.

The data of an object can be accessed only by the functions associated with that object.

When a program is executed, the objects interact by sending messages to one another through functions.

Continued …

Objects



Example

Basic Concepts of OOP

continue …

Classes

- *Classes are user-defined data types.*
- A class is a structure of making a user-defined data type which holds the entire set of data of an object.
- So, a class is basically a blueprint or a template of a set of objects which shares some common properties and behaviours.
- It is only an logical component and not the physical entity.
- *A class is a collection of objects of similar type.*

Continued …

- Once a class is created, we can create any number of objects belonging to that class.
- Actually, a class doesn’t specify any data, it determines the properties or methods that may contain in an object of that class.
- Classes normally act like a built-in data type, but they are actually a user-defined data types.

Continued…

For example,

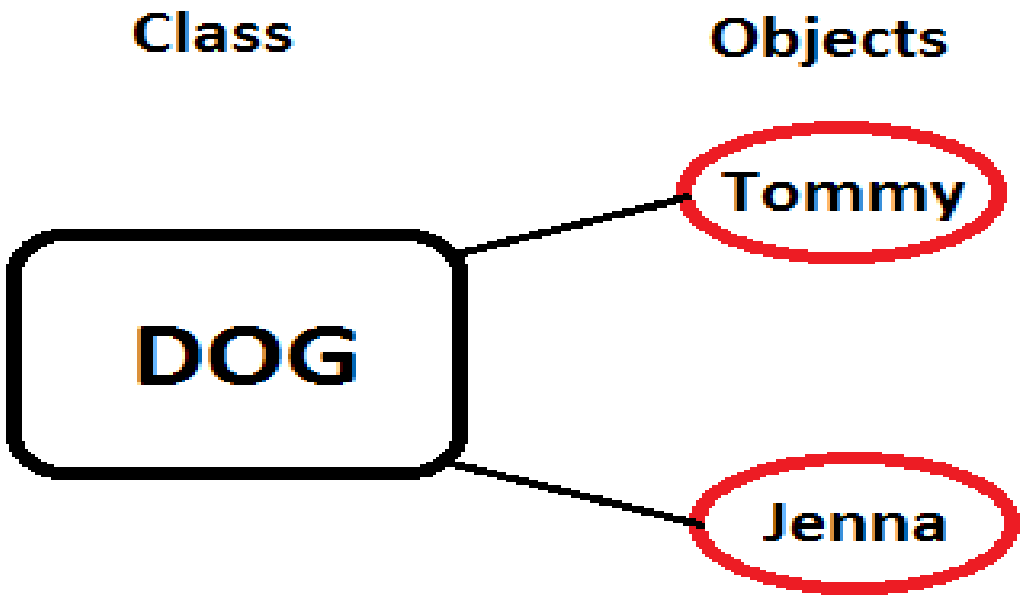
if you had a class called “Expensive Cars” it could have objects like Mercedes, BMW, Toyota, etc. Its properties(data) can be price or speed of these cars. While the methods may be performed with these cars are driving, reverse, braking etc.

Continued …

For Example,

‘Dog’ is a class which holds properties like ‘dog name’, ‘age of the dog’, ‘weight of the dog’ etc. Now, if we have two dogs named ‘Tommy’ & ‘Jenna’, then they will have their own properties like name, age & weight; which may be different for both the dog but they are similar in types. So, the two dogs ‘Tommy’ & ‘Jenna’ will be called as objects of the class ‘Dog’. Here the class ‘Dog’ is representing the common properties of both the two objects named as ‘Tommy’ & ‘Jenna’.

Continued …



continue …

Message Passing

- An OOP consists of a set of objects that communicate with each other.
- OOP involves the following steps:
  - ✓ Creating classes that define objects and their behavior.
  - ✓ Creating objects from class definitions.
  - ✓ Establishing communication among objects.
- Objects communicate with one another by sending and receiving information.

continue …

Message Passing

- A message for an object is a request for execution of a procedure.
- The receiving object will invoke a function and generates results.
- Message passing involves specifying:
  - The name of the Object.
  - The name of the Function.
  - The information to be send.

Basic Concepts of OOP

continue …

Data Abstraction

Data abstraction is a concept **which hides the background details and represents only the needed information to the outside world**. It is a technique of creating a new data type that is suited for a specific application.

For example, You are riding a bike, You only know the essential methods to ride a bike like accelerating, braking, how to change the gears, using the clutch etc. but while riding you do you get into internal details of the bike like gearbox, fuel-injection or carburetor, workings of the motor, engine blocks etc ? You just use the accelerator and your bike starts moving, You don’t need to know the details of what happening inside to ride the bike.

This is called abstraction, you only know the essential parts to ride a bike without including the background process or explanation.

Basic Concepts of OOP

continue …

Data Abstraction

Since the classes use the concept of data abstraction, they are known as Abstracted Data Types (ADT).

Basic Concepts of OOP

continue …

Encapsulation

- The wrapping up of data and functions into a single unit is known as encapsulation.
- The data is not accessible to the outside world, and only those functions which are wrapped in the class can access it.
- i.e. If you want to read a data in an object, you have to call the member function in the object. The function will read the data and return the data back to you. So, you have no access to the data directly. Since data is hidden, it is secured from accidental alteration.
- These functions provide the interface between the object's data and the program. This insulation of the data from direct access by the program is called data hiding or information hiding.

The attributes wrapped in the classes are called data members and the functions that operate on these data are called methods or member functions.

Basic Concepts of OOP

continue …

Inheritance

- It is basically a method **which provides a way that capabilities and properties from one class to come into another class**.
- New class shares common characteristics with the class from which it is derived.
- Here the new class is called as the derived class and the existing class i.e. the class from which the new class is derived is called as the base class.

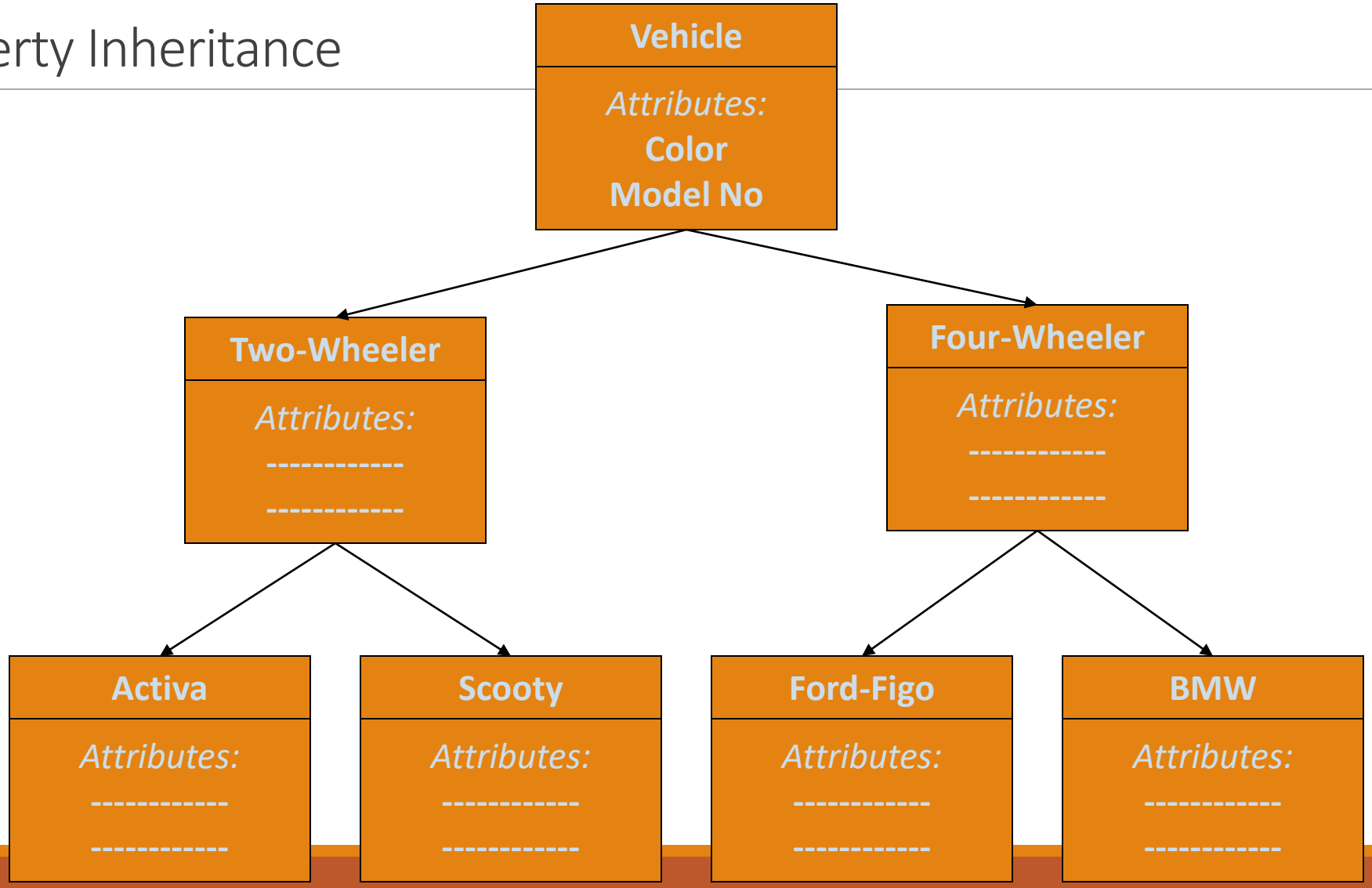
continue …

Inheritance

- Inheritance provides the idea of reusability.
  - We can add additional features to an existing class without modifying it.

    (By deriving new class from existing one. The new class will have the combined features of both the classes.)

Property Inheritance



Basic Concepts of OOP

continue …

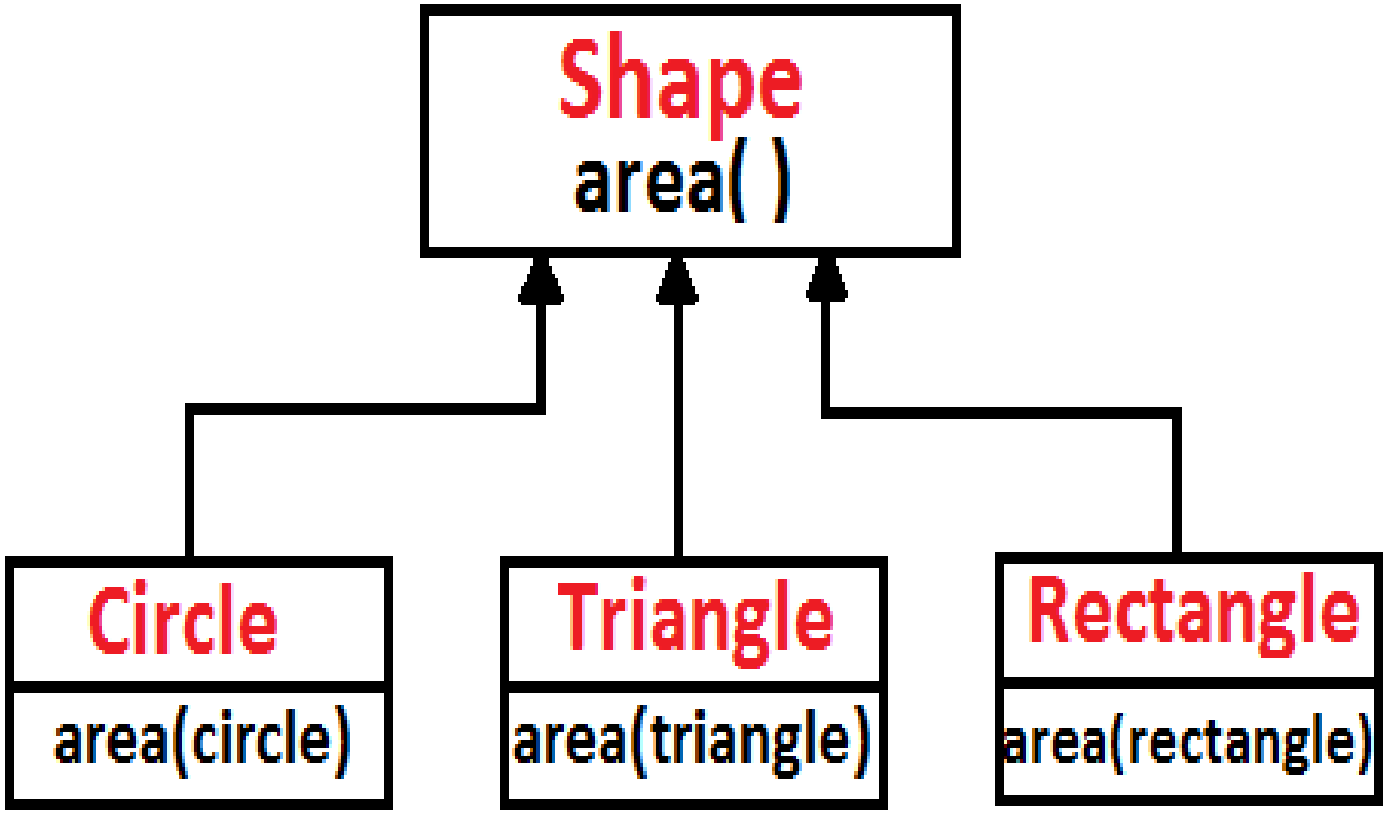
Polymorphism *- ability to take more than one form*

polymorphism is the ability for a message or data to be processed in more than one form.

It is an important concept of object oriented programming which supports the capability of an object of a class to behave differently in response to a message or action.

For a real-life example, consider yourself. At the same time, you may have some different characteristics like a student, a guitarist, a football player. This is called polymorphism, since you have some different forms or characteristics with respect to a different action or situation.

Continued …


Shape
area( )
Circle
area(circle)
Triangle
area(triangle)
Rectangle
area(rectangle)

Continued …

- The process of making an operator to exhibit different behaviours in different instances is known as operator overloading.
- << Insertion Operator
- << Left-shift bit-wise operator
- Using a single function name to perform different types of tasks is known as function overloading.
- add(3, 5) gives 8
- Add(“hello”, “-world”) gives “hello-world”

continue …

Dynamic Binding

- Binding refers to the linking of a procedure call to the code to be executed in response to the call.
- Dynamic binding (late binding) means that the code associated with a given procedure call is not known until the time of the call at run-time.
- It is associated with polymorphism and inheritance.

Benefits of OOP

The principal advantages are:

- Through inheritance, we can eliminate redundant code and extend the use of existing classes.
- We can build programs from the standard working modules that communicate with one another, rather than having to start writing the code from scratch. This leads to saving of development time and higher productivity.
- The principle of data hiding helps the programmer to build secure programs that cannot be invaded by code in other parts of the program.
- It is possible to have multiple instances of an object to co-exist without any inter- ference.

Benefits of OOP

continue …

- It is possible to map objects in the problem domain to those in the program. It is easy to partition the work in a project based on objects.
- The data-centered design approach enables us to capture more details of a model in implementable form.
- Object-oriented systems can be easily upgraded from small to large systems.
- Message passing techniques for communication between objects makes the interface descriptions with external systems much simpler.
- Software complexity can be easily managed.

Applications of OOP

continue …

- Real-time systems Simulation and modelling
- Object-oriented databases
- Hypertext, hypermedia and expertext
- AI and expert systems
- Neural networks and parallel programming
- Decision support and office automation systems
- CIM/CAM/CAD systems

Difference Between POP & OOP

Feature / Attributes	Procedure-Oriented Programming (POP)	Object-Oriented Programming (OOP)
Focus	Functions and procedures	Objects and classes
Data Handling	Data is separate from functions; functions manipulate global data	Data and functions are encapsulated within objects (data hiding)
Modularity	Less modular, functions can be tightly coupled	More modular, through classes and encapsulation
Reusability	Code duplication is common	Inheritance and polymorphism promote reusability
Maintenance	Can be difficult due to tight coupling	Easier to maintain due to encapsulation and modularity
Approach	Top-down approach (focus on procedures)	Bottom-up approach (focus on objects and their interactions)

Difference Between POP & OOP

Feature	Procedure-Oriented Programming (POP)	Object-Oriented Programming (OOP)
Access Control	No access modifiers, data is often globally accessible	Access modifiers (public, private, protected) control access to data and methods
Real-world Modeling	Less relatable to real-world scenarios	More closely models real-world entities and their interactions
Polymorphism	Not supported	Supported through method overloading and overriding
Inheritance	Not supported	Supported
Example Languages	C, FORTRAN	C++, Java, Python

Contents are taken from various resources for educational purpose only.